

Eventim Disability Integration

Barrierefreie Abwicklung von Ticketbestellungen & Nachteilsausgleichsanträgen

Version 1.0 – 23. Juli 2025

Eventim Disability Integration

Dieses Repository enthält eine Next.js Anwendung samt Express Backend, mit der Eventim eine barrierefreie Abwicklung von Ticketbestellungen und eine Verwaltung von Nachteilsausgleichsanträgen ermöglicht. Das Projekt gliedert sich in ein Frontend und einen Node.js Server im Unterordner `eventim-disability-integration`.

Inhaltsverzeichnis

- Einleitung (Soheil)
 - Projekthintergrund und Motivation
 - Aktuelle Herausforderungen:
 - Gesellschaftlicher und rechtlicher Kontext:
- Projektvision und Zielsetzung
 - Primäre Ziele:
 - Sekundäre Ziele:
- Zielgruppen und Stakeholder
 - Primäre Nutzergruppen
 - Interne Stakeholder
- Projektübersicht (Soheil)
- Setup
- Architektur und eingesetzte Technologien
- Datenbank
 - Tabellen im Detail
- Backend-Endpunkte
- Seitenübersicht
- Rollen
- Komponenten und Hooks
- Weiterführende Hinweise
 - Rollen und Fähigkeiten
 - Fehlertoleranz des Backends
 - Potenzielle Verbesserungen
- Nächste Schritte

Einleitung (Soheil)

Projekthintergrund und Motivation

Die Eventim-Plattform ist eine der führenden Ticketing-Lösungen im deutschsprachigen Raum und verarbeitet jährlich Millionen von Ticket-Transaktionen für Konzerte, Festivals, Theater und Sportveranstaltungen. Trotz dieser marktführenden Position existiert eine bedeutende Lücke in der Barrierefreiheit und digitalen Inklusion für Menschen mit Behinderungen.

Aktuelle Herausforderungen:

- Menschen mit Schwerbehindertenausweis müssen ihre Tickets derzeit telefonisch buchen
- Nachteilsausgleiche (reduzierte Preise, kostenfreie Begleitpersonen) werden nicht automatisch angewendet
- Keine digitale Verifizierung von Behindertenausweisen möglich
- Manuelle Bearbeitung führt zu langen Wartezeiten und Fehlerquellen
- Schlechte User Experience für eine vulnerable Zielgruppe

Gesellschaftlicher und rechtlicher Kontext:

In Deutschland leben etwa 7,9 Millionen Menschen mit einer anerkannten Schwerbehinderung (Stand 2021). Diese Personengruppe hat nach dem Sozialgesetzbuch IX (SGB IX) und verschiedenen Landesgesetzen Anspruch auf Nachteilsausgleiche bei kulturellen Veranstaltungen. Die EU-Richtlinie zur Barrierefreiheit (European Accessibility Act) verpflichtet zudem bis 2025 zur digitalen Barrierefreiheit von Ticketing-Systemen.

Projektvision und Zielsetzung

Vision: "Eventim wird zur ersten vollständig inklusiven Ticketing-Plattform, die Menschen mit Behinderungen den gleichen komfortablen, digitalen Zugang zu kulturellen Erlebnissen ermöglicht wie allen anderen Nutzern."

Primäre Ziele:

- Vollständige Integration der Disability-Features in die bestehende Plattform
- Eliminierung manueller Prozesse durch intelligente Verifizierung
- Erfüllung aller rechtlichen Anforderungen Erschließung einer neuen Zielgruppe mit signifikantem Marktpotential

Sekundäre Ziele:

- Reduzierung der Support-Anfragen um 70%
- Steigerung der Kundenzufriedenheit in der Zielgruppe
- Positionierung als Accessibility-Leader im Ticketing-Markt

Zielgruppen und Stakeholder

Primäre Nutzergruppen

Menschen mit Schwerbehinderung (Primary Users)

- Anzahl: ~7,9 Millionen in Deutschland
- Charakteristika: Diverse Behinderungsarten, unterschiedliche technische Affinität
- Bedürfnisse: Einfache, barrierefreie Buchung, automatische Vergünstigungen
- Pain Points: Aktuelle telefonische Buchung, Wartezeiten, Unsicherheit über Berechtigung

Begleitpersonen und Angehörige (Secondary Users)

- Anzahl: ~2–3 Millionen potentielle Nutzer
- Charakteristika: Oft technisch versierter, buchen stellvertretend
- Bedürfnisse: Transparente Abrechnung, klare Berechtigungsregeln

Interne Stakeholder

Service-Mitarbeiter

- Anzahl: ~50 Personen im Customer Service
- Bedürfnisse: Effiziente Tools zur Antragsbearbeitung, reduzierte Anruflast
- Erfolgsmetriken: 60% weniger Disability-bezogene Tickets

Admin-User (Content-Manager, Venue-Manager)

- Anzahl: ~20 Power-User
- Bedürfnisse: Einfache Konfiguration von barrierefreien Bereichen
- Erfolgsmetriken: Reduzierte Setup-Zeit für neue Events

Projektübersicht (Soheil)

Das Eventim Disability Integration System ist eine umfassende Erweiterung der bestehenden Ticketing-Plattform, die Menschen mit Behinderungen vollständige digitale Autonomie beim Ticket-Kauf ermöglicht. Das System integriert sich nahtlos in die bestehende Eventim-Architektur und erweitert diese um spezialisierte Funktionen für Barrierefreiheit und Nachteilsausgleiche.

Kernfunktionalitäten:

- **Intelligente Behindertenausweis-Verifizierung**

- Upload-Interface für Vorder- und Rückseite des Schwerbehindertenausweises
 - OCR-basierte automatische Datenextraktion (Grad, Merkzeichen, Gültigkeit)
 - Manuelle Nachbearbeitung durch geschulte Service-Mitarbeiter
 - Echtzeit-Validierung gegen bekannte Ausweis-Muster
- **Automatisierte Nachteilsausgleiche**
 - Dynamische Preisanpassungen basierend auf Behinderungsgrad und Merkzeichen
 - Automatische Begleitpersonen-Integration für B-Merkzeichen-Inhaber
 - Priorisierte Anzeige barrierefreier Sitzplätze und Bereiche
 - Transparente Darstellung aller Vergünstigungen im Checkout

Setup

1. Repository klonen und in das Projekt wechseln:

```
git clone <repo-url>
cd EventimDisabilityIntegration/eventim-disability-integration
```

2. Abhängigkeiten installieren:

```
npm install
```

3. Datenbankzugang konfigurieren:

Legen Sie im Verzeichnis `server` eine Datei `credentials.json` an. Beispiel:

```
{
  "host": "152.53.119.113",
  "port": 5433,
  "user": "postgres",
  "password": "example",
  "database": "db1",
  "sessionSecret": "example-secret"
}
```

4. Entwicklungsumgebung starten:

```
npm run dev
```

Damit starten sowohl das Next.js Frontend auf <http://localhost:3000> als auch das Backend unter <http://localhost:4000>.

Architektur und eingesetzte Technologien

Die Anwendung besteht aus einem **Next.js** Frontend und einem **Express** Backend. Als Datenbank kommt **PostgreSQL** zum Einsatz. Die Wahl fiel auf diese Kombination, da sie leichtgewichtig, gut erweiterbar und auch ohne großen Konfigurationsaufwand lokal ausführbar ist. Next.js liefert die React basierte Oberfläche und kann sowohl statische Seiten als auch serverseitig gerenderte Inhalte bereitstellen. Express dient als schlanker REST-API Server, der über die `server`-Ordnerstruktur umgesetzt ist. Die Kommunikation zwischen Frontend und Backend erfolgt ausschließlich über JSON-basierte HTTP-Aufrufe. Alle Benutzerdaten werden in PostgreSQL gespeichert. Für Passwort-Hashes wird **bcrypt** verwendet, Uploads werden mit **multer** direkt in der Datenbank gespeichert. In der Entwicklung laufen beide Dienste parallel über `npm run dev`.

Datenbank

Die Datenbank basiert auf PostgreSQL und wird komplett über `server/backup_script.sql` erstellt. Enthalten sind Tabellen für Benutzer, Rollen, Künstler, Touren, Events, Veranstaltungsorte sowie Tabellen zur Abwicklung von Bestellungen und zur Speicherung von Disability-Merkmalen. Das Schema ist in der folgenden Mermaid-Grafik visualisiert.

Tabellen im Detail

Nachfolgend eine kurze Beschreibung jeder Tabelle und ihrer Beziehungen.

Tabelle	Zweck / wichtige Spalten	Abhängigkeiten
countries	Länder mit ISO-Code.	–

Tabelle	Zweck / wichtige Spalten	Abhängigkeiten
cities	Städte innerhalb eines Landes.	<code>countries</code> via <code>country_id</code>
user_roles	Definiert Rechte einer Rolle.	–
users	Registrierte Personen samt Adresse und optionalen Disability-Angaben.	<code>user_roles</code> via <code>role</code>
artists	Künstlerinformationen.	–
genres / subgenres	Klassifikation von Touren.	<code>genres</code> via <code>genre_id</code>
tours	Übergeordnete Tour einer Reihe von Events.	–
tour_artists	Zuordnung Künstler ↔ Tour.	<code>artists</code> , <code>tours</code>
tour_genres / tour_subgenres	Genre-Verknüpfungen einer Tour.	<code>tours</code> , <code>genres / subgenres</code>
venues	Veranstaltungsorte.	<code>cities</code> via <code>city_id</code>
areas	Sitzplatz- bzw. Zugänglichkeitsbereiche.	–
venue_areas	Bereiche innerhalb eines Venues.	<code>venues</code> , <code>areas</code>
events	Konkrete Veranstaltungstermine.	<code>tours</code> , <code>venues</code>
event_categories	Ticket-Kategorien (Preis, Support für Behinderte).	<code>events</code> via <code>event_id</code>
event_supporting_acts	Support-Acts eines Events.	<code>events</code> , <code>artists</code>
event_venue_areas	Zuordnung Event ↔ Bereich mit Kapazität.	<code>events</code> , <code>venue_areas</code> , <code>event_categories</code>
images	Speicherung von hochgeladenen Bildern.	Beliebige Entität über <code>entity_type / entity_id</code>
carts	Aktiver Warenkorb eines Nutzers.	<code>users</code>
cart_items	Einzelne Artikel im Warenkorb.	<code>carts</code> , <code>events</code> , <code>event_categories</code>
checkouts	Zwischenschritt zwischen Warenkorb und Bestellung.	<code>users</code>

Tabelle	Zweck / wichtige Spalten	Abhängigkeiten
checkout_items	Positionen des Checkouts inkl. Preis.	checkouts , events , event_categories
payment_options / shipping_options	Stammdaten für Zahlungs- und Versandarten.	–
orders	Abgeschlossene Bestellungen.	users , payment_options
tickets	Konkrete Ticketdatensätze.	orders , event_categories
order_tickets	Relation zwischen Order und Ticket (1-n).	orders , tickets
disability_marks	Mögliche Merkmale auf Behindertenausweisen.	areas via area_id
user_disability_marks	Zuordnung User ↔ Marks.	users , disability_marks

Die Fremdschlüssel schützen vor inkonsistenten Daten. Viele Tabellen nutzen UUIDs als Primärschlüssel. Preise werden als `NUMERIC(10,2)` gespeichert, was zwei Nachkommastellen erlaubt. Für einfache Lookups existieren diverse Join-Tabellen (z. B. `tour_genres`).

Backend-Endpunkte

Die wichtigsten Routen des Express-Servers (siehe `server/server.js`) sind:

Methode	Pfad	Beschreibung
POST	<code>/register-user</code>	Registrierung eines neuen Nutzers inkl. optionaler Behindertenausweis-Daten
POST	<code>/login-user</code>	Benutzeranmeldung, legt Session-Cookie an
GET	<code>/session-status</code>	Prüft, ob ein Nutzer eingeloggt ist und liefert Profilinformationen
POST	<code>/logout</code>	Beendet die aktuelle Session
GET	<code>/user-address</code>	Liefert die hinterlegte Anschrift des eingeloggten Nutzers
GET	<code>/disability-marks</code>	Auflistung möglicher Markierungen des Behindertenausweises

Methode	Pfad	Beschreibung
GET	/pending-disability-requests	Offene Anträge auf Nachteilsausgleich für den Service-Bereich
POST	/disability-requests/:id/accept	Antrag eines Nutzers akzeptieren
POST	/disability-requests/:id/decline	Antrag eines Nutzers ablehnen
GET	/artists	Auflistung aller Künstler
POST	/create-artist	Neuen Künstler anlegen
GET	/tours-detailed	Touren inkl. Events und Zugänglichkeitsdaten
POST	/create-tour	Neue Tour anlegen
GET	/venues-detailed	Liste aller Veranstaltungsorte mit Areas
POST	/create-venue	Neuen Veranstaltungsort anlegen
POST	/cart-items	Ticket zur Warenkorb-Session hinzufügen
GET	/checkout	Aktuellen Checkout laden
POST	/orders	Bestellung aus abgeschlossenem Checkout erzeugen

Dies ist nur ein Auszug. Weitere Endpunkte finden sich direkt im Quellcode von `server/server.js`.

Seitenübersicht

Die Next.js Anwendung befindet sich unter `src/pages` und nutzt dynamische Routen. Wichtige Seiten sind:

Pfad	Zweck / Inhalte
/	Startseite mit Highlights sowie Künstler- und Tourübersicht
/artists/[artist]	Detailseite eines Künstlers und Übersicht zugehöriger Touren
/artists/[artist]/[tour]	Informationen und Events einer Tour
/artists/[artist]/[tour]/[event]	Detailseite eines Events mit Buchungsoption

Pfad	Zweck / Inhalte
<code>/registration</code>	Formular zur Kontoerstellung und Erfassung optionaler Behindertenausweis-Daten
<code>/login</code>	Anmelden bzw. Schnellanmeldung inkl. Passwort-Reset
<code>/profile</code>	Persönliche Daten, Bestellungen und eigene Events verwalten
<code>/checkout</code>	Mehrstufiger Bestellprozess (Versanddaten, Zahlung, Abschluss)
<code>/admin</code>	Einstieg in alle Admin-Unterseiten zur Pflege von Künstlern, Ländern, Genres, Touren und Veranstaltungsorten
<code>/service</code>	Zugriffspunkt für Service-Mitarbeiter (u.a. Nachteilsausgleichsanträge und Account-Management)

Alle Seiten unter `/admin/*` und `/service/*` setzen entsprechende Berechtigungen voraus.

Rollen

Es existieren drei grundlegende Benutzerrollen:

Rolle	Berechtigungen
User	Kann Tickets kaufen, eigenes Profil bearbeiten und Nachteilsausgleich beantragen.
Service	Darf Service-Seiten nutzen, z.B. Anträge prüfen oder Accounts verwalten.
Admin	Umfasst sämtliche Rechte: Erstellung/Bearbeitung von Daten, Rollenvergabe und Zugriff auf alle Admin-Seiten.

Die Zuordnung der Rechte erfolgt über die Tabelle `user_roles`. Beim Registrieren erhält jeder Nutzer standardmäßig die Rolle **User**.

Komponenten und Hooks

Der Quellcode unter `src` gliedert sich in wiederverwendbare React-Komponenten und mehrere Custom Hooks:

Komponenten (Auswahl)

Datei	Einsatz
nav-bar.jsx	Hauptnavigation mit Suche und Warenkorb-Anzeige
footer.jsx	Gemeinsamer Seitenfuß
LoadingOverlay.jsx	Überblendet die Seite während Daten geladen werden
DisabilityRequestModal.jsx	Dialog zur Beantragung eines Nachteilsausgleichs
DeleteAccountModal.jsx	Bestätigungsdialog zum Löschen des Accounts
smallArtistCard.jsx / smallTourCard.jsx	Vorschaukarten auf der Startseite

Hooks

Hook	Zweck
useAuth	Kümmert sich um Login-Status und hält Userdaten clientseitig vor
useCart	Verwaltet den Warenkorb und lädt Items vom Server
useRequireAccess	Leitet unberechtigte Nutzer auf die Login-Seite um
useValidation	Hilft bei Formularvalidierungen

Die Komponenten sind so aufgebaut, dass sie in verschiedenen Seiten wiederverwendet werden können. Die Hooks stellen gemeinsam genutzte Logik bereit und erleichtern die Anbindung an das Backend.

Weiterführende Hinweise

- Das Frontend erwartet als API-Basis `NEXT_PUBLIC_API_URL` (Standard: `http://localhost:4000`).
- Für Datei-Uploads wird `multer` verwendet. Bilder werden in der Tabelle `images` gespeichert und über `/image/:id` ausgeliefert.
- Ein Cronjob im Server entfernt veraltete Warenkörbe und Checkouts alle 60 Sekunden.

Rollen und Fähigkeiten

Die Tabelle `user_roles` definiert Berechtigungen. Aktuell existieren drei Rollen:

Rollenname	Beschreibung	Edit	Create	Appoint Roles	Account Mgmt	Disability Approval	Delete
user	Regulärer Eventim Nutzer	false	false	false	false	false	false
service	Service-Mitarbeiter von Eventim	false	false	false	true	true	false
admin	Vollzugriff auf alle Funktionen	true	true	true	true	true	true

Fehlertoleranz des Backends

- `server/db.js` überwacht die Datenbankverbindung und versucht bei Fehlern einen Reconnect. Ein Circuit-Breaker auf Basis von `opossum` verhindert Kaskadenfehler.
- `server/server.js` startet erst, wenn eine DB-Verbindung besteht, und führt regelmäßige Aufräumbjobs aus.
- Der Server wird mit **PM2** im Cluster-Modus betrieben und startet bei Fehlern automatisch neu.
- Ungefangene Fehler führen zu einem kontrollierten Exit, damit PM2 unmittelbar einen Neustart durchführen kann.

Potenzielle Verbesserungen

- HTTPS/TLS für alle Verbindungen erzwingen.
- Stärkere Validierung der Eingaben und Rate-Limiting für API-Calls.
- Zentrales Logging (ELK-Stack o.Ä.) für die PM2-Prozesse einbinden.

Testkonzept

User-Tests

Testfall-ID	Testfall-Beschreibung	Aktion/Schritte	Tatsächliches Ergebnis
TC01	URL-Test: Zugriff auf Admin-Bereich als Normalnutzer	Als normaler Nutzer über URL `/admin/...` auf Admin-Seite navigieren	Umleitung auf Anmeldeseite
TC02	Löschtest: Stadt bei bestehendem Event in dieser Stadt	Versuch, eine Stadt zu löschen in der es ein Event gibt	Popup unterhalb der Darstellung, "Fehler beim Löschen da Städte dieses Landes in einem Stadion verwendet werden"
TC03	Löschtest: Venue bei bestehendem Event in dieser Venue	Versuch, eine Venue zu löschen in dem es ein Event gibt	Popup unterhalb der Darstellung, "Venue kann nicht gelöscht werden, da Tickets für Events existieren"
TC04	Löschtest: Land bei bestehendem Event in diesem Land	Versuch, ein Land zu löschen in dem es ein Event gibt	Popup unterhalb der Darstellung, "Fehler beim Löschen da Städte dieses Landes in einem Stadion verwendet werden"
TC05	Löschtest: Nutzerlöschen aus Usermaske bei bestehender Buchung eines Events	Versuch, einen Nutzer (aus User-Oberfläche) zu löschen bei einer bestehenden Buchung eines Events	Schrift unterhalb der Darstellung, "Sie können Ihr Konto nicht löschen, da sie Tickets für Veranstaltungen haben, die noch nicht stattgefunden haben."
TC06	Löschtest: Event mit vorhandenen Tickets	Versuch, ein Event zu löschen, für das bereits Tickets verkauft wurden	Nicht möglich. Keine Option zum Löschen von Events mit Tickets
TC07	Löschtest: Tour mit Events und Tickets	Versuch, eine Tour zu löschen, die Events mit verkauften Tickets enthält	Nicht möglich. Keine Option zum Löschen von Events mit Tickets
TC08	Löschtest: Artist mit Event	Versuch, einen Artist zu löschen	Artist sind im Insgesamten nicht Löschar.
TC09	Löschtest: Genre mit Event	Versuch, ein Genre zu löschen, für das bereits ein Ticket eines Events existiert	Popup unterhalb der Darstellung, "Fehler beim Löschen des Genres. Möglicherweise ist dieses Genre mit einer Tour verbunden."
TC10	Löschtest: Sub-Genre mit Event	Versuch, ein Genre zu löschen	Popup unterhalb der Darstellung, "Fehler beim Löschen des Sub-Genres. Möglicherweise ist dieses Sub-Genre mit einer Tour verbunden."
TC11	Mehr Tickets von Usern bestellen als Tickets vorhanden.	Event erstellt, bei dem es nur ein Ticket gibt. Versucht für dieses Event mehr Tickets zu kaufen als vorhanden	Bei der Maximalen Ticketzahl (die in dem Fall durch die Anzahl der Tickets maximiert ist) graut sich der Kauf-Button aus. Damit ist das tatsächliche Kaufen eine Tickets nicht mehr möglich.
TC12	Löschtest: User löscht eigenen account obwohl ein Offener Schwerbehindertenantrag besteht	Versuch, ein User zu Löschen der einen offenen Schwerbehinderten Antrag hat	Antrag wird aus Liste der offenen Anträge gelöscht.
TC13	Für ein Event was angelegt wird, keine Sitzplätze anbieten (ob "normal" oder für Schwerbehinderte ist hier egal)	Versuch, ein Event in einer Venue mit Behindertenbereich anzulegen und keine Tickets für Schwerbehinderte anzubieten sollte nicht erlaubt sein	Nicht erlaubt. Kategorien/Sitzplätze die hinzugefügt werden müssen einen Wert von >0 haben. Damit werden gesetzliche Vorgaben eingehalten.
TC14	Für ein Event mehrere Schwerbehindertentickets kaufen (ob Gratis begleitperson oder Sondertickets)	Versuch, mehrere Tickets zu kaufen, die mit einem Status eines Schwerbehinderten erlangt werden.	Nicht möglich. "In den Warenkorb"-Button nicht möglich anzuklicken. Dieser wird Ausgegraut.

Als Normalnutzer wurde zunächst versucht, durch direkte Eingabe der URL „/admin/...“ auf den Admin-Bereich zuzugreifen (TC01). Das System leitete jedoch konsequent auf die Anmeldeseite um. Anschließend folgte eine Reihe von Löschttests: Beim Versuch, eine Stadt zu entfernen, in der bereits ein Event existiert (TC02), erschien ein Popup mit dem Hinweis, dass das Löschen fehlschlug, weil Städte dieses Landes in einem Stadion verwendet werden. Gleiches geschah beim Löschen einer Venue mit bestehendem Event (TC03); hier meldete das System, die Venue könne nicht entfernt werden, da Tickets für Events existieren. Auch das Löschen eines Landes, in dem Events stattfinden (TC04), wurde mit einem entsprechenden Fehler-Popup verhindert, das erneut auf verwendete Städte verwies.

Der Versuch, einen Nutzer direkt über die Benutzeroberfläche zu löschen, obwohl noch Buchungen für zukünftige Veranstaltungen bestehen (TC05), scheiterte ebenfalls: Unterhalb der Darstellung erschien der Text, das Konto könne nicht gelöscht werden, solange Tickets für noch nicht stattgefundene Events vorhanden sind. Ein Event mit bereits verkauften Tickets zu löschen (TC06) war gar nicht erst vorgesehen – die Option existierte schlicht nicht; dasselbe galt für das Entfernen einer Tour, die Events mit verkauften Tickets enthält (TC07). Artists ließen sich generell nicht löschen (TC08). Beim Versuch, ein Genre zu entfernen, für das bereits Tickets eines Events existieren (TC09), zeigte das System ein Popup, das einen Fehler beim Löschen meldete und als mögliche Ursache eine Tour-Verknüpfung nannte. Identisch

verhielt es sich beim Sub-Genre (TC10): Auch hier verhinderte ein Fehlerhinweis das Löschen und verwies auf eine mögliche Verbindung zu einer Tour.

Ein weiterer Test zielte darauf ab, mehr Tickets zu bestellen, als tatsächlich vorhanden waren (TC11). Wurde ein Event mit nur einem Ticket angelegt und der Nutzer versuchte, mehr zu kaufen, graute sich der Kauf-Button bei Erreichen der maximalen Ticketanzahl aus, sodass ein weiterer Erwerb unmöglich war. Interessant war auch der Fall, in dem ein Nutzer sein Konto löschen wollte, obwohl noch ein offener Schwerbehindertenantrag bestand (TC12): Statt den Löschvorgang zu blockieren, entfernte das System den Antrag aus der Liste offener Anträge.

Beim Anlegen eines Events in einer Venue mit Behindertenbereich wurde geprüft, ob sich Sitzplätze – egal ob regulär oder für Schwerbehinderte – komplett weglassen lassen (TC13). Das System erlaubte dies nicht: Kategorien bzw. Sitzplätze müssen einen Wert größer null haben, womit gesetzliche Vorgaben eingehalten werden. Schließlich wurde getestet, ob mehrere Schwerbehindertentickets (etwa Sondertickets oder Gratis-Begleitpersonen) gleichzeitig gekauft werden können (TC14). Auch das war nicht möglich: Der „In den Warenkorb“-Button blieb ausgegraut und ließ sich nicht anklicken.

Nächste Schritte

Das Grundgerüst funktioniert lokal stabil: Registrierung, Login, Rollenverwaltung und der Bestellprozess sind lauffähig. Das Projekt ist in Frontend und Backend klar getrennt und setzt auf PostgreSQL als persistente Datenbasis.

Vor einer finalen Bereitstellung sollten noch folgende Punkte bearbeitet werden:

1. **Konfiguration bereinigen** – Secrets in Umgebungsvariablen auslagern und Produktionsbuilds automatisieren.
2. **Security prüfen** – HTTPS erzwingen, CORS-Regeln präzisieren und optionale 2FA vorsehen.
3. **Fehlertoleranz weiter ausbauen** – zentrales Logging und Alerting einrichten.

Bereits sehr gut funktionieren das Rollenmodell, die Bild-Uploads sowie die periodische Datenbereinigung im Backend.

Anhang

Datenbank-Modell

```
@startuml
!theme plain
```

```
top to bottom direction
skinparam linetype ortho

class areas {
  name: varchar(50)
  description: varchar(100)
  disability_category_for: char(3)
  id: uuid
}
class artists {
  name: varchar(255)
  biography: text
  website: varchar(255)
  created_at: timestamp with time zone
  updated_at: timestamp with time zone
  artist_image: uuid
  id: uuid
}
class cart_items {
  cart_id: uuid
  event_id: uuid
  event_category_id: uuid
  quantity: integer
  added_at: timestamp with time zone
  is_assistance_ticket: boolean
  id: uuid
}
class carts {
  user_id: uuid
  created_at: timestamp with time zone
  updated_at: timestamp with time zone
  id: uuid
}
class checkout_items {
  checkout_id: uuid
  event_category_id: uuid
  quantity: integer
  price: numeric(10,2)
  added_at: timestamp with time zone
  event_id: uuid
  is_assistance_ticket: boolean
  id: uuid
}
class checkouts {
  user_id: uuid
  created_at: timestamp with time zone
  updated_at: timestamp with time zone
  id: uuid
}
class cities {
  name: varchar(100)
  country_id: uuid
  id: uuid
}
class countries {
  name: varchar(100)
```

```
    iso_code: char(2)
    id: uuid
}
class disability_marks {
  description: varchar(100)
  area_id: uuid
  mark_code: char(3)
}
class event_categories {
  event_id: uuid
  name: text
  price: numeric(10,2)
  disability_support_for: char(3)
  id: uuid
}
class event_supporting_acts {
  event_id: uuid
  artist_id: uuid
}
class event_venue_areas {
  event_id: uuid
  venue_area_id: uuid
  capacity: integer
  category_id: uuid
  id: uuid
}
class events {
  tour_id: uuid
  venue_id: uuid
  description: text
  created_at: timestamp with time zone
  updated_at: timestamp with time zone
  start_time: timestamp with time zone
  end_time: timestamp with time zone
  door_time: timestamp with time zone
  id: uuid
}
class genres {
  name: varchar(50)
  id: uuid
}
class images {
  image_data: bytea
  image_type: text
  entity_type: text
  entity_id: uuid
  id: uuid
}
class order_tickets {
  order_id: uuid
  ticket_id: uuid
  id: uuid
}
class orders {
  user_id: uuid
  created_at: timestamp with time zone
```



```
    street_address: varchar(255)
    postal_code: varchar(20)
    city: varchar(100)
    country: varchar(100)
    is_paid: boolean
    salutation: varchar(20)
    first_name: varchar(100)
    last_name: varchar(100)
    company: varchar(255)
    payment_option_id: uuid
    id: uuid
}
class payment_options {
    label: varchar(50)
    description: varchar(100)
    icon_src: varchar(50)
    id: uuid
}
class shipping_options {
    label: varchar(100)
    price: numeric
    description: varchar(100)
    id: uuid
}
class subgenres {
    genre_id: uuid
    name: text
    id: uuid
}
class tickets {
    order_id: uuid
    event_category_id: uuid
    seat_number: varchar(50)
    price: numeric(10,2)
    created_at: timestamp with time zone
    is_assistance_ticket: boolean
    id: uuid
}
class tour_artists {
    tour_id: uuid
    artist_id: uuid
}
class tour_genres {
    tour_id: uuid
    genre_id: uuid
}
class tour_subgenres {
    tour_id: uuid
    subgenre_id: uuid
}
class tours {
    title: varchar(255)
    subtitle: varchar(255)
    start_date: date
    end_date: date
    created_at: timestamp with time zone
```

```
    updated_at: timestamp with time zone
    tour_image: uuid
    id: uuid
  }
class user_disability_marks {
  user_id: uuid
  mark_code: char(3)
}
class user_roles {
  name: varchar(50)
  description: varchar(100)
  has_editing_access: boolean
  has_creation_access: boolean
  has_role_appointing_capability: boolean
  has_account_management_access: boolean
  has_disability_approval_access: boolean
  has_deletion_permission: boolean
  id: uuid
}
class users {
  salutation: varchar(20)
  first_name: varchar(100)
  last_name: varchar(100)
  company: varchar(255)
  street_address: varchar(255)
  postal_code: varchar(20)
  city: varchar(100)
  country: varchar(100)
  email: varchar(255)
  phone: varchar(20)
  birth_date: date
  request_for_disability: boolean
  disability_degree: integer
  disability_card_image_front: uuid
  created_at: timestamp with time zone
  updated_at: timestamp with time zone
  password: text
  disability_card_image_back: uuid
  is_currently_disabled: boolean
  disability_card_expiry_date: date
  role: uuid
  visible_user_id: integer
  user_id: uuid
}
class venue_areas {
  venue_id: uuid
  max_capacity: integer
  area_id: uuid
  id: uuid
}
class venues {
  name: varchar(255)
  address: varchar(500)
  city_id: uuid
  website: varchar(255)
  created_at: timestamp with time zone
```

```

    updated_at: timestamp with time zone
    venue_image: uuid
    id: uuid
  }

  cart_items      -[#595959,plain]-^  carts
  : "cart_id:id"
  cart_items      -[#595959,plain]-^  event_categories
  : "event_category_id:id"
  cart_items      -[#595959,plain]-^  events
  : "event_id:id"
  carts           -[#595959,plain]-^  users
  : "user_id"
  checkout_items  -[#595959,plain]-^  checkouts
  : "checkout_id:id"
  checkout_items  -[#595959,plain]-^  event_categories
  : "event_category_id:id"
  checkout_items  -[#595959,plain]-^  events
  : "event_id:id"
  checkouts       -[#595959,plain]-^  users
  : "user_id"
  cities          -[#595959,plain]-^  countries
  : "country_id:id"
  disability_marks -[#595959,plain]-^  areas
  : "area_id:id"
  event_categories -[#595959,plain]-^  events
  : "event_id:id"
  event_supporting_acts -[#595959,plain]-^  artists
  : "artist_id:id"
  event_supporting_acts -[#595959,plain]-^  events
  : "event_id:id"
  event_venue_areas -[#595959,plain]-^  event_categories
  : "category_id:id"
  event_venue_areas -[#595959,plain]-^  events
  : "event_id:id"
  event_venue_areas -[#595959,plain]-^  venue_areas
  : "venue_area_id:id"
  events          -[#595959,plain]-^  tours
  : "tour_id:id"
  events          -[#595959,plain]-^  venues
  : "venue_id:id"
  order_tickets   -[#595959,plain]-^  orders
  : "order_id:id"
  order_tickets   -[#595959,plain]-^  tickets
  : "ticket_id:id"
  orders          -[#595959,plain]-^  payment_options
  : "payment_option_id:id"
  orders          -[#595959,plain]-^  users
  : "user_id"
  subgenres       -[#595959,plain]-^  genres
  : "genre_id:id"
  tickets         -[#595959,plain]-^  event_categories
  : "event_category_id:id"
  tickets         -[#595959,plain]-^  orders
  : "order_id:id"
  tour_artists    -[#595959,plain]-^  artists

```

```
: "artist_id:id"
tour_artists      -[#595959,plain]-^  tours
: "tour_id:id"
tour_genres       -[#595959,plain]-^  genres
: "genre_id:id"
tour_genres       -[#595959,plain]-^  tours
: "tour_id:id"
tour_subgenres    -[#595959,plain]-^  subgenres
: "subgenre_id:id"
tour_subgenres    -[#595959,plain]-^  tours
: "tour_id:id"
user_disability_marks -[#595959,plain]-^  disability_marks
: "mark_code"
user_disability_marks -[#595959,plain]-^  users
: "user_id"
users            -[#595959,plain]-^  user_roles
: "role_id"
venue_areas      -[#595959,plain]-^  areas
: "area_id:id"
venue_areas      -[#595959,plain]-^  venues
: "venue_id:id"
venues          -[#595959,plain]-^  cities
: "city_id:id"
@enduml
```